

Technical Architecture Specification

Badminton Motion Intelligence Platform

Working title: Badminton Motion Lab

Document version: 0.1

Status: Draft for implementation

Architecture model: Local-first, cloud-light, offline-capable

Initial target environment: Windows desktop, Chromium browser, Python local worker

Mandatory recurring cloud cost: Rp0/month

Related document: Product Requirements Document v0.1

1. Purpose

Dokumen ini mendefinisikan arsitektur teknis untuk Badminton Motion Intelligence Platform, termasuk:

- Batas tanggung jawab setiap komponen.
- Deployment topology.
- Komunikasi web dan local worker.
- Pemrosesan video lokal.
- Penyimpanan data dan artifact.
- Job orchestration.
- Model adapter.
- Offline operation.
- Sinkronisasi cloud.
- Security boundaries.
- Cost guardrails.
- Observability.
- Migration path menuju cloud, multi-camera, dan full-match analysis.

Arsitektur harus mendukung visi produk jangka panjang tanpa memaksa seluruh fitur dibangun sejak rilis awal.

2. Architecture Objectives

Arsitektur harus memenuhi tujuan berikut.

2.1 Zero Mandatory Cloud Compute

Pemrosesan video, pose estimation, rendering, dan perhitungan metrik dapat berjalan sepenuhnya pada komputer pengguna.

2.2 Local Video Ownership

Video asli tidak wajib meninggalkan perangkat pengguna.

2.3 Cloud as Control Plane

Cloud digunakan terutama untuk:

- Authentication.
- Session metadata.
- Job state.
- Metric summaries.
- Configuration.
- Analysis history.
- Optional synchronization.

Cloud tidak menjadi execution plane wajib.

2.4 Offline-Capable Analysis

Analisis tetap dapat dimulai, diteruskan, dan diselesaikan ketika internet atau Supabase tidak tersedia.

2.5 Reproducible Analysis

Setiap hasil dapat ditelusuri ke:

- Source media.
- Pipeline version.
- Model version.
- Configuration.
- Input artifact.
- Runtime environment.
- Metric definition.
- Human correction.

2.6 Replaceable Intelligence

Pose, racket, shuttle, temporal detection, biomechanics, dan assessment harus dapat diganti secara independen.

2.7 Progressive Complexity

Arsitektur yang sama harus mendukung:

- Satu stroke.

- Beberapa repetisi.
- Satu sesi latihan.
- Multi-camera session.
- Full match.
- Coach workspace.
- Research benchmark.

2.8 Cost-Aware by Design

Tidak ada layanan berbayar yang menjadi dependency untuk alur utama.

3. Core Architectural Decisions

ADR-001 — Local-First Execution

Semua proses berat berjalan pada local worker secara default.

Cloud worker dapat ditambahkan kemudian dengan mengimplementasikan kontrak worker yang sama.

ADR-002 — Cloud Control Plane, Local Execution Plane

Sistem dibagi menjadi:

Control Plane

- Web application.
- Authentication.
- Session metadata.
- Job metadata.
- Analysis summaries.
- Device registration.
- Optional artifact synchronization.

Execution Plane

- Local file access.
- FFmpeg processing.
- Computer vision.
- Machine-learning inference.
- Metric calculation.
- Overlay rendering.
- Artifact packaging.

ADR-003 — Video Is Not a Database Asset

Video bukan record database. Video adalah media asset yang memiliki:

- Unique identifier.
- Fingerprint.
- Storage locator.
- Ownership.
- Metadata.
- Availability status.

Locator dapat menunjuk ke:

- Local path.
- Local object store.
- Supabase Storage.
- Cloudflare R2.
- Future S3-compatible storage.

ADR-004 — Supabase Stores Metadata, Not Dense Frame Data

Supabase PostgreSQL menyimpan:

- Sessions.
- Captures.
- Analysis runs.
- Events.
- Summary metrics.
- Findings.
- Annotation revisions.
- Model and pipeline registry.

Supabase tidak menyimpan satu row untuk setiap landmark pada setiap frame.

Dense data seperti pose time series, racket trajectory, dan shuttle trajectory disimpan dalam artifact files.

ADR-005 — Local Artifact Package Is the Portable Source of Truth

Setiap analysis run menghasilkan satu versioned analysis package yang dapat:

- Disimpan lokal.
- Diimpor ulang.
- Dipindahkan ke komputer lain.
- Disinkronkan ke storage.
- Digunakan untuk re-analysis.
- Digunakan sebagai benchmark input.

ADR-006 — Polling Before Realtime

Local worker menggunakan durable job polling dan database lease sebagai mekanisme utama.

Supabase Realtime dapat digunakan untuk memperbarui UI lebih cepat, tetapi bukan satu-satunya sumber state. Supabase mendukung Broadcast, Presence, dan Postgres Changes; namun pesan Broadcast tidak dirancang sebagai durable history yang dapat selalu diputar ulang. Karena itu, status final tetap disimpan di database.

ADR-007 — Deterministic Metric Engine

Nilai metrik dihitung oleh versioned deterministic functions.

LLM hanya menerima structured findings setelah metric engine selesai.

ADR-008 — Raw and Corrected Data Remain Separate

Model output, transformed output, dan human correction tidak saling menimpa.

ADR-009 — Every Artifact Has a Hash

Artifact menggunakan SHA-256 atau algoritma hash yang setara untuk:

- Integrity checking.
- Cache reuse.
- Deduplication.
- Reproducibility.
- Idempotency.

ADR-010 — Browser File Handles Are Not Canonical Storage

Browser dapat memperoleh akses ke file lokal melalui user-mediated file APIs, tetapi persistent file access tidak dijadikan fondasi utama. Local worker tetap menjadi pemilik proses file dan penyimpanan analysis package.

ADR-011 — Local Worker Is a Product Component

Local worker bukan temporary development script. Ia memiliki:

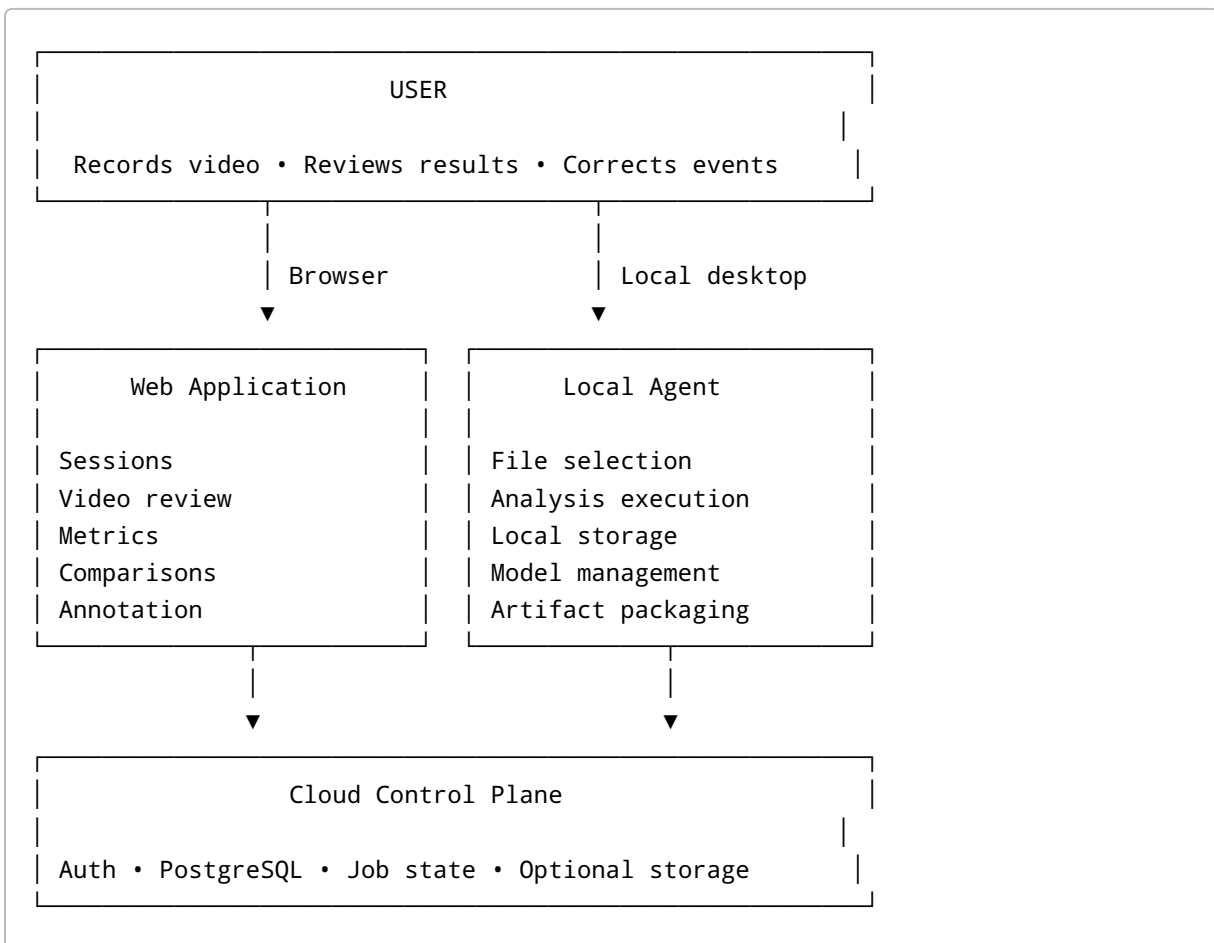
- Version.
- Device identity.
- Capability profile.
- Local database.
- Update mechanism.
- Health state.

- Job contract.
- Diagnostics.

ADR-012 — Cloud Migration Uses the Same Worker Contract

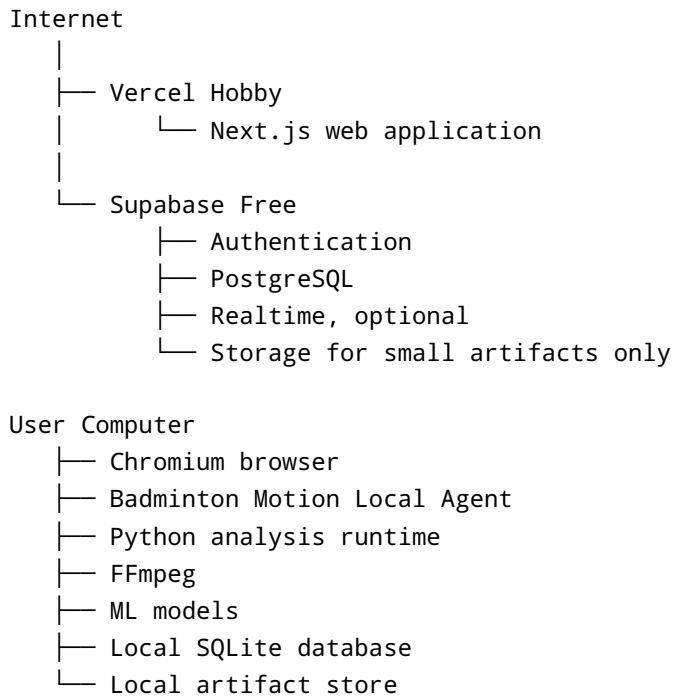
Future cloud GPU worker mengimplementasikan interface yang sama dengan local worker sehingga pipeline tidak perlu ditulis ulang.

4. System Context



5. Deployment Topology

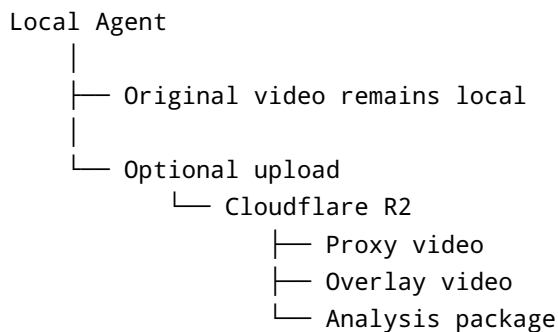
5.1 Initial Rp0 Topology



Vercel Hobby saat ini ditujukan untuk proyek personal dan aplikasi berskala kecil serta tersedia tanpa biaya dasar. Arsitektur tidak menggunakan Vercel Functions untuk inference atau video processing.

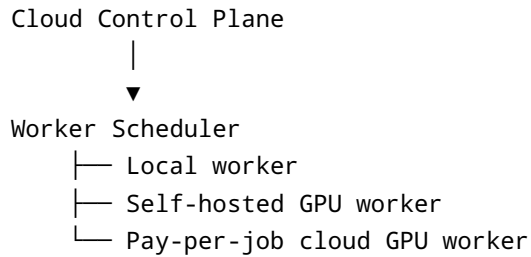
Supabase Free saat ini membatasi database pada 500 MB sebelum proyek masuk read-only mode. Storage Free memiliki kuota 1 GB dan batas upload file maksimal 50 MB. Batas ini memperkuat keputusan untuk tidak menyimpan video asli maupun frame-level pose data di Supabase.

5.2 Optional Object Storage Topology



Cloudflare R2 Standard memiliki free usage berupa 10 GB-month storage, satu juta Class A operations, sepuluh juta Class B operations, dan free internet egress. Namun R2 tetap opsional karena pemakaian di atas free tier dapat menimbulkan biaya.

5.3 Future Cloud Compute Topology



Worker selection berdasarkan:

- Privacy preference.
- Model requirement.
- Video availability.
- Hardware compatibility.
- User-approved cost.
- Queue priority.

6. Component Architecture

6.1 Web Application

Responsibilities

- Authentication.
- Athlete and session management.
- Analysis history.
- Worker-device management.
- Job creation.
- Result visualization.
- Video review.
- Event correction.
- Metric comparison.
- Report generation.
- Cloud synchronization configuration.

Non-Responsibilities

Web application tidak melakukan:

- Full-resolution video transcoding.
- Heavy pose inference.
- Shuttle tracking.
- Multi-camera reconstruction.
- Large overlay rendering.
- Long-running pipeline orchestration.

Recommended Stack

- Next.js.
- TypeScript.
- Tailwind CSS.
- shadcn/ui.
- Supabase client.
- TanStack Query.
- Zod.
- Canvas 2D for initial overlay.
- WebGL only when Canvas becomes a measured bottleneck.

Application Layers

```
apps/web/  
├─ app/  
├─ features/  
│   ├─ athletes/  
│   ├─ sessions/  
│   ├─ captures/  
│   ├─ analysis-runs/  
│   ├─ viewer/  
│   ├─ annotation/  
│   ├─ comparisons/  
│   └─ devices/  
├─ components/  
├─ lib/  
│   ├─ supabase/  
│   ├─ contracts/  
│   ├─ auth/  
│   └─ telemetry/  
└─ workers/  
    └─ browser-pose.worker.ts
```

6.2 Cloud Control Plane

Responsibilities

- User identity.
- Authorization.
- Cloud metadata.
- Device registration.
- Durable job records.
- Analysis-run registry.
- Summary metrics.
- Optional notifications.
- Optional cloud artifact locators.

Primary Services

- Supabase Auth.
- Supabase PostgreSQL.
- Supabase Row Level Security.
- Supabase Realtime as optional UI enhancement.
- Supabase Storage for small assets only.

Data Stored in PostgreSQL

Suitable:

- User profile.
- Session metadata.
- Capture metadata.
- Analysis-run metadata.
- Job states.
- Pipeline steps.
- Event summaries.
- Metric observations.
- Findings.
- Comparison records.
- Annotation revisions.
- Model registry.
- Artifact manifests.

Not suitable:

- Raw video.
- Extracted frame collection.
- One database row per landmark per frame.
- Full-resolution segmentation masks.
- Large tensors.
- Model weights.

- Uncompressed trajectories.
-

6.3 Local Agent

Local Agent adalah desktop-side application yang bertindak sebagai gateway antara pengguna, file lokal, analysis engine, dan cloud control plane.

Responsibilities

- Device registration.
- User authentication or device authorization.
- Local file selection.
- Media fingerprinting.
- Local media catalogue.
- Job polling.
- Pipeline execution.
- Progress updates.
- Local artifact management.
- Result packaging.
- Optional result synchronization.
- Model installation and validation.
- Diagnostics.

Initial Form

Fase awal dapat menggunakan:

- Python CLI.
- Localhost FastAPI service.
- Minimal local control page.
- System tray wrapper kemudian.

Recommended Evolution

Phase 1:
Python CLI + localhost service

Phase 2:
Packaged Windows executable

Phase 3:
System tray application

Phase 4:
Optional Tauri desktop shell

Localhost Binding

Service hanya bind ke:

127.0.0.1

bukan:

0.0.0.0

Default port:

8765

Port harus dapat dikonfigurasi bila terjadi konflik.

Example Local Endpoints

```
GET /health
GET /version
GET /capabilities
POST /files/select
POST /sessions/link
POST /jobs/run
POST /jobs/{id}/cancel
GET /jobs/{id}
GET /artifacts/{id}/manifest
```

Endpoint ini tidak boleh menjadi remote public API.

6.4 Analysis Engine

Analysis Engine menjalankan versioned pipeline steps.

Modules

```
workers/local-analyzer/
├─ agent/
├─ orchestration/
├─ media/
```

```
|— quality/
|— calibration/
|— perception/
|   |— person/
|   |— pose/
|   |— racket/
|   |— shuttle/
|— temporal/
|— reconstruction/
|— metrics/
|— assessment/
|— rendering/
|— packaging/
|— diagnostics/
```

Execution Profiles

Fast

- Reduced input resolution.
- Frame sampling.
- Lightweight pose model.
- No shuttle tracking.
- No expensive rendering.

Standard

- Full target frame rate where possible.
- Standard pose model.
- Court calibration.
- Primary metrics.
- Basic overlay.

Detailed

- Higher-resolution inference.
- Racket and shuttle modules.
- Full smoothing.
- More metrics.
- Detailed overlay.

Research

- Custom model.
- No automatic cleanup.
- Raw intermediate outputs retained.
- Benchmark metadata required.

6.5 Local Metadata Store

Use SQLite sebagai local operational database.

Responsibilities

- Local media catalogue.
- Local sessions.
- Pending jobs.
- Job execution states.
- Artifact index.
- Cloud-sync queue.
- Model registry.
- Cached cloud metadata.
- Device settings.

Why SQLite

- No separate service.
- Transactional.
- Easy backup.
- Supported by Python.
- Suitable for one-device operation.
- Allows offline operation.

Local Database Is Not a Cache Only

Local database menjadi durable record untuk local analysis. Cloud synchronization adalah layer tambahan.

6.6 Local Artifact Store

Default Windows location:

`%LOCALAPPDATA%\BadmintonMotionLab`

Suggested structure:

```
BadmintonMotionLab/  
├─ config/  
├─ database/  
│   └─ local.db  
├─ media/  
└─ references/
```

```
|   └─ proxies/
├─ models/
├─ runs/
|   └─ {analysis_run_id}/
├─ cache/
├─ logs/
└─ exports/
```

User may configure a different data directory, particularly when the system drive has limited space.

7. Analysis Package Specification

Every analysis run produces one directory or ZIP-compatible bundle.

7.1 Suggested Layout

```
{analysis_run_id}.bml/
├─ manifest.json
├─ environment.json
├─ media/
|   ├── source-reference.json
|   ├── metadata.json
|   ├── proxy.mp4
|   └─ thumbnail.webp
├─ quality/
|   └─ quality-report.json
├─ calibration/
|   ├── court.json
|   ├── camera.json
|   └─ transforms.json
├─ perception/
|   ├── person-tracks.parquet
|   ├── pose-raw.parquet
|   ├── pose-smoothed.parquet
|   ├── racket-track.parquet
|   └─ shuttle-track.parquet
├─ temporal/
|   ├── rallies.json
|   ├── strokes.json
|   └─ movement-events.json
├─ metrics/
|   ├── observations.json
|   └─ summaries.json
```

```
├─ assessments/
│   └─ findings.json
├─ annotations/
│   └─ initial-model-output.json
├─ renders/
│   └─ overlay.mp4
└─ checksums.sha256
```

Not every run must contain every file.

7.2 Manifest Requirements

```
{
  "schema_version": "1.0.0",
  "analysis_run_id": "uuid",
  "session_id": "uuid",
  "capture_id": "uuid",
  "created_at": "ISO-8601",
  "pipeline": {
    "name": "standard-single-camera",
    "version": "0.1.0",
    "git_commit": "commit-sha"
  },
  "source": {
    "media_fingerprint": "sha256",
    "storage_type": "local_reference",
    "original_included": false
  },
  "artifacts": [],
  "quality_flags": [],
  "completion_status": "complete_with_warnings"
}
```

7.3 Dense Data Formats

Recommended:

- Parquet for time-series tabular data.
- JSON for manifests, events, definitions, and summaries.
- MP4 for video.
- WebP or JPEG for evidence frames.
- NPZ only for internal model intermediate data.
- CSV only for export, not canonical storage.

7.4 Bundle Security

During import:

- Reject absolute paths.
 - Reject `../` traversal.
 - Enforce maximum expanded size.
 - Validate manifest schema.
 - Validate every checksum.
 - Reject unexpected executables.
 - Never execute code from an imported package.
 - Treat imported model configuration as data only.
-

8. Media Ingestion Architecture

8.1 Preferred Initial Flow

1. User creates a session in web application.
2. Session receives a short pairing code or QR code.
3. User opens Local Agent.
4. Local Agent links to the session.
5. User selects video through Local Agent.
6. Agent fingerprints and inspects the video.
7. Agent creates local capture record.
8. Capture metadata is synchronized to cloud.
9. Analysis is started.

File selection is performed by Local Agent rather than relying on a persistent browser file handle.

8.2 Fingerprinting

Use a two-stage approach.

Fast Fingerprint

Derived from:

- File size.
- Modification time.
- Selected beginning and ending byte ranges.
- Media duration.
- Video stream metadata.

Used for quick catalogue matching.

Strong Fingerprint

SHA-256 of the full file.

Used before:

- Benchmark registration.
- Final reproducibility export.
- Deduplication.
- External synchronization.

8.3 Video Normalization

Original video remains untouched.

Normalized derivatives may standardize:

- Orientation.
- Pixel format.
- Codec.
- Constant frame rate.
- Audio sampling.
- Timestamp base.

Example normalized target:

```
Container: MP4
Video: H.264
Pixel format: yuv420p
Frame rate: source-equivalent constant frame rate
Audio: AAC or removed when unnecessary
```

Normalization configuration must be stored in the manifest.

8.4 Proxy Generation

Proxy video is used by web viewer.

Typical profile:

- 720p maximum.
- Lower bitrate.
- Constant frame rate.

- Same visual orientation.
- Same timeline duration.
- No frame trimming unless recorded in a time mapping.

8.5 Frame Identity

Every frame-level observation uses:

- `frame_index`.
- `timestamp_us`.
- `source_timebase`.
- `normalized_frame_index`.
- `normalized_timestamp_us`.

Do not rely only on frame number when input may use variable frame rate.

9. Worker Registration and Authentication

9.1 Device Identity

Each Local Agent installation receives:

- `device_id`.
- Human-readable device name.
- Device public key.
- Worker version.
- Capability profile.
- Last-seen timestamp.
- Revocation state.

9.2 Recommended Pairing Flow

1. Local Agent generates a public/private key pair.
2. Agent shows a short-lived pairing code.
3. User opens the web application and enters the code.
4. Web application confirms the signed-in user.
5. Cloud associates device public key with user account.
6. Agent signs future requests with its private key.
7. User can revoke the device from Settings.

Private key is stored through the operating-system credential store where possible.

9.3 Phase-One Simplification

Before public-key device authentication is implemented, the developer build may use:

- Personal access token.
- User-scoped worker token.
- Environment variable.
- OS keyring storage.

The Supabase `service_role` key must never be stored in Local Agent, browser code, or repository.

9.4 Worker Scopes

Example scopes:

```
worker:jobs:read  
worker:jobs:claim  
worker:jobs:update  
worker:artifacts:create  
worker:metrics:write  
worker:device:heartbeat
```

Worker must not automatically receive permission to:

- Delete users.
- Read unrelated sessions.
- Change billing.
- Access other users' videos.
- Bypass all RLS policies.

10. Job Orchestration

10.1 Job Model

Each `analysis_job` contains:

```
id  
user_id  
session_id  
capture_id  
analysis_run_id  
pipeline_key
```

```
pipeline_version
priority
status
required_capabilities
configuration
lease_owner
lease_expires_at
attempt_count
max_attempts
cancel_requested_at
created_at
started_at
finished_at
```

10.2 Job States

```
draft
queued
claimed
running
paused
cancel_requested
cancelled
complete
complete_with_warnings
failed
superseded
```

10.3 Job Claiming

Worker performs atomic job claim through database RPC.

Conceptual behavior:

```
select next compatible queued job
for update skip locked;

update job
set
  status = 'claimed',
  lease_owner = current_device,
  lease_expires_at = now() + lease_duration;
```

10.4 Lease

Recommended initial lease:

5 minutes

Worker renews lease periodically during processing.

If lease expires:

- Job is not immediately duplicated.
- System checks the last heartbeat.
- User may resume, retry, or explicitly release it.
- Non-idempotent steps require cleanup before retry.

10.5 Heartbeat

Worker sends heartbeat containing:

- Device state.
- Current job.
- Current step.
- Progress.
- Available disk.
- Memory status.
- GPU status.
- Worker version.

Heartbeat need not be more frequent than required. An initial interval of 30–60 seconds is sufficient.

10.6 Durable State Versus UI Events

Durable state:

- Stored in PostgreSQL or local SQLite.
- Used for recovery.
- Used as source of truth.

Transient progress:

- May use Supabase Realtime.
- May use local WebSocket.
- Can be dropped without corrupting job state.

10.7 Cancellation

Cancellation occurs at safe boundaries.

Each step declares:

- Interruptible.
- Interruptible with cleanup.
- Non-interruptible.

The worker checks `cancel_requested_at` :

- Between frame batches.
- Between pipeline steps.
- Before expensive rendering.
- Before artifact synchronization.

10.8 Retry Policy

Retry categories:

Automatically Retryable

- Temporary file lock.
- Temporary network failure.
- Optional cloud-sync failure.
- Recoverable model-loading failure.

Retryable After User Action

- Insufficient disk.
- Missing source file.
- Unsupported GPU driver.
- Moved local media.

Not Automatically Retryable

- Corrupt media.
 - Invalid calibration.
 - Schema incompatibility.
 - Missing required model.
 - Bundle checksum failure.
-

11. Pipeline Orchestration

11.1 Pipeline as DAG

Pipeline is represented as a directed acyclic graph.

```
inspect_media
├─ normalize_media
└─ technical_quality
    │
    └─ normalize_media
        ├── visual_quality
        ├── player_detection
        └─ court_calibration
            │
            └─ player_detection
                └─ pose_estimation
                    └─ pose_smoothing
                        ├── temporal_events
                        └─ metrics
court_calibration
├─ coordinate_transform
└─ metrics
temporal_events
└─ metrics
metrics
├─ assessments
└─ rendering
```

11.2 Step Contract

Each step implements:

```
class PipelineStep:
    key: str
    version: str
    inputs: list[ArtifactRequirement]
    outputs: list[ArtifactDefinition]
    resources: ResourceRequirement

    def validate_inputs(self, context) -> ValidationResult:
        ...
```



```
def run(self, context) -> StepResult:
    ...

def validate_outputs(self, result) -> ValidationResult:
    ...
```

11.3 Idempotency Key

```
hash(
    step_key
    + step_version
    + input_artifact_hashes
    + model_version
    + normalized_configuration
)
```

When matching output exists and passes checksum validation, the step may reuse it.

11.4 Partial Reprocessing

Example:

- User corrects contact frame.
- Pose does not need rerun.
- Event-derived metrics rerun.
- Assessments rerun.
- Report rerenders.

Dependency graph determines invalidated outputs.

11.5 Pipeline Profiles

Profiles are configuration manifests rather than hardcoded branches.

Example:

```
pipeline: standard-single-camera
version: 0.1.0

steps:
  inspect_media:
    enabled: true

  normalize_media:
```

```
    enabled: true

pose_estimation:
  enabled: true
  adapter: rtmpose
  model: body-medium

shuttle_tracking:
  enabled: false

metric_engine:
  metric_set: single_camera_core_v1
```

12. Model Adapter Architecture

12.1 Adapter Principle

Domain layer must not understand model-specific tensors.

Each model adapter converts native output into canonical contracts.

12.2 Pose Adapter Contract

```
Input:
- Media artifact.
- Frame/time range.
- Person track.
- Model configuration.

Output:
- Pose sequence.
- Landmark definition.
- Landmark observations.
- Confidence.
- Runtime metadata.
```

12.3 Racket Adapter Contract

```
Input:
- Media.
- Player crop or full frame.
```

- Dominant hand.
- Optional wrist track.

Output:

- Racket head coordinates.
- Handle coordinates.
- Shaft orientation.
- Confidence.
- Observed/interpolated status.

12.4 Shuttle Adapter Contract

Input:

- Media.
- Court crop.
- Frame range.
- Optional rally range.

Output:

- Shuttle coordinate by timestamp.
- Detection confidence.
- Occlusion state.
- Interpolation state.

12.5 Temporal Adapter Contract

Input:

- Pose sequence.
- Racket track.
- Shuttle track.
- Audio events.
- Player position.

Output:

- Rally proposals.
- Stroke proposals.
- Movement phases.
- Contact proposals.
- Confidence.

12.6 Model Registry

Each model record stores:

```
model_id
model_family
model_name
model_version
task
framework
license
source
weight_checksum
landmark_schema
supported_devices
minimum_runtime_version
installation_status
validation_status
```

12.7 Model Installation

Model download must:

- Use allowlisted source.
- Validate expected checksum.
- Store license information.
- Support manual installation.
- Never execute arbitrary downloaded scripts.
- Separate model weights from application updates.

13. Coordinate System Architecture

13.1 Supported Coordinate Systems

Image Pixel

```
origin: top-left
x: pixels rightward
y: pixels downward
```

Normalized Image

```
x: 0-1
y: 0-1
```

Court Plane

```
origin: defined court reference
x: court width direction
y: court length direction
unit: meter
```

Camera Coordinates

```
origin: camera optical center
unit: meter or scale-relative
```

Body-Local Coordinates

Relative to body reference such as:

- Hip center.
- Shoulder line.
- Dominant side.
- Body height.

Reconstructed World 3D

Created from:

- Multi-camera triangulation.
- Calibrated reconstruction.
- Validated monocular estimate with explicit labeling.

13.2 Coordinate Record

Every coordinate sequence references:

```
coordinate_system_id
unit
transform_id
source_artifact_id
calibration_id
confidence
```

13.3 Transform Graph

Transforms form a graph:

```
raw pixel
  ↓ orientation transform
normalized pixel
  ↓ homography
court plane

raw pixel
  ↓ camera calibration
camera ray
  ↓ multi-view triangulation
world 3D
```

13.4 Transform Versioning

Changing court corner correction creates a new calibration revision.

Dependent results become stale but are not deleted.

14. Metric Engine Architecture

14.1 Metric Definition

```
interface MetricDefinition {
    key: string
    version: string
    name: string
    unit: string
    requiredInputs: InputRequirement[]
    supportedCaptureModes: CaptureMode[]
    calculationMethod: string
    limitations: string[]
}
```

14.2 Metric Observation

```
interface MetricObservation {
    id: string
    metricKey: string
    metricVersion: string
    analysisRunId: string
    entityType: "session" | "rally" | "stroke" | "phase"
```

```

entityId: string
value: number | null
unit: string
confidence: number | null
uncertainty?: {
  lower: number
  upper: number
  method: string
}
evidence: EvidenceReference[]
qualityFlags: string[]
status:
  | "measured"
  | "estimated"
  | "insufficient_data"
  | "unsupported"
}

```

14.3 Calculation Isolation

Metric function receives canonical input only.

It must not:

- Query model-native files directly.
- Infer missing values silently.
- Access UI state.
- Generate coaching language.
- Change source artifacts.

14.4 Missing Data Policy

Missing input leads to one of:

- `insufficient_data`.
- `unsupported`.
- `estimated` with explicit method.
- Metric omitted from assessment.

Zero must never be used as a substitute for missing data.

14.5 Numerical Precision

UI precision is based on validated accuracy.

Example:

- Internal angle: 147.384°.
 - Display before validation: 147°.
 - Do not display 147.38° when model error is several degrees.
-

15. Assessment Engine Architecture

15.1 Inputs

Assessment receives:

- Metric observations.
- Capture profile.
- Quality flags.
- Athlete baseline.
- Selected rubric.
- Comparison session.
- Validation metadata.

15.2 Rule Example

```
finding_key: delayed_recovery
rubric_version: personal_baseline_v1

when:
  metric: recovery_duration
  comparison: greater_than
  reference: personal_baseline_p75
  minimum_confidence: 0.75

output:
  severity: medium
  category: recovery
  template_key: delayed_recovery_personal
```

15.3 Findings Are Versioned

Every finding stores:

- Assessment version.
- Source metrics.
- Rule.

- Reference.
- Evidence.
- Confidence.
- Limitation.
- Human review state.

15.4 LLM Boundary

LLM receives findings only after assessment.

LLM output is stored separately as:

generated_explanation

not as:

measurement

16. Offline-First Architecture

16.1 Offline Operations

Without internet, user can:

- Select local video.
- Create local session.
- Run analysis.
- Review local report.
- Export analysis package.
- Correct events.
- Compare locally available sessions.

16.2 Local IDs

Use globally unique identifiers generated locally.

Recommended:

UUIDv7

This avoids waiting for cloud-generated IDs and improves chronological indexing.

16.3 Sync Outbox

Every local mutation intended for cloud creates an outbox record:

```
id
entity_type
entity_id
operation
payload
created_at
attempt_count
last_attempt_at
sync_status
```

16.4 Sync States

```
local_only
pending_upload
syncing
synced
conflict
failed
deleted_locally
pending_remote_delete
```

16.5 Conflict Strategy

Append-Only Data

Analysis runs, annotations, and job events use append-only records. Conflict probability is low.

Editable Metadata

Session title and notes use:

- `updated_at`.
- Revision number.
- Last-writer detection.
- Manual conflict resolution when both sides changed.

Never Merge Automatically

Do not automatically merge:

- Court calibration.
- Contact frame correction.
- Stroke event boundary.
- Metric override.

Each becomes a separate revision.

17. Cloud Data Synchronization

17.1 Default Sync Set

Automatically sync:

- Session metadata.
- Capture metadata.
- Analysis-run manifest summary.
- Pipeline status.
- Metric summaries.
- Findings.
- Annotation metadata.
- Small evidence thumbnails.

Do not automatically sync:

- Raw video.
- Proxy video.
- Full pose sequence.
- Extracted frames.
- Overlay video.
- Model weights.
- Dense trajectories.

17.2 Optional Artifact Sync

User selects:

- Metadata only.
- Metadata and thumbnails.
- Metadata and analysis package.
- Include proxy video.
- Include overlay video.

- Include original video.

17.3 Availability States

Every asset has:

```
local_available  
cloud_available  
remote_only  
missing  
moved  
deleted  
checksum_mismatch
```

17.4 Re-Linking Moved Video

When local path changes:

1. Agent reports source unavailable.
2. User chooses replacement file.
3. Agent computes fingerprint.
4. Strong or compatible fingerprint is compared.
5. Asset locator is updated.
6. Existing analysis history remains.

18. Database Architecture

18.1 Schemas

Recommended logical schemas:

```
public  
auth-derived profile data  
  
core  
athletes, sessions, captures  
  
processing  
jobs, runs, steps, artifacts, workers  
  
analysis  
events, metrics, findings
```

```
review
annotations, revisions, decisions

research
datasets, benchmarks, validation

registry
models, pipelines, metric definitions, rubrics
```

Supabase API exposure must be reviewed before placing tables outside the default schema.

18.2 High-Level Tables

worker_devices

```
id
user_id
name
public_key
worker_version
capabilities
last_seen_at
revoked_at
created_at
```

analysis_jobs

```
id
user_id
session_id
capture_id
analysis_run_id
pipeline_key
configuration
status
required_capabilities
lease_owner
lease_expires_at
attempt_count
error_summary
created_at
updated_at
```

analysis_runs

```
id
user_id
session_id
capture_id
parent_run_id
pipeline_key
pipeline_version
status
capture_mode
manifest_hash
local_availability
cloud_availability
started_at
finished_at
created_at
```

processing_artifacts

```
id
analysis_run_id
artifact_type
schema_version
content_hash
byte_size
storage_type
storage_locator
availability
created_at
```

metric_observations

```
id
analysis_run_id
entity_type
entity_id
metric_definition_id
value_numeric
value_json
unit
confidence
status
quality_flags
```

```
evidence_json
created_at
```

18.3 Avoid Frame-Level PostgreSQL Explosion

Example risk:

```
33 body landmarks
× 3,600 frames
× 10 sessions
= 1,188,000 landmark observations
```

Storing every observation as an individual PostgreSQL row is unnecessary for the initial architecture.

Instead:

- Store pose sequence in Parquet.
- Store summary, index, and artifact locator in PostgreSQL.
- Extract selected evidence points into metric records.

18.4 Row Level Security

Every user-owned table requires policy based on:

```
auth.uid() = user_id
```

Derived access through organization or coach relationship must be explicit.

Do not use client-side filtering as authorization.

19. Security Architecture

19.1 Trust Boundaries

```
Untrusted:
- Uploaded/imported media
- Imported analysis packages
- Browser input
- Model output
- LLM output
```

- External model files

Semi-trusted:

- Local Agent
- Registered user device

Trusted:

- Versioned application code
- Validated metric engine
- Server-side authorization logic

19.2 Local Agent Security

- Bind only to loopback.
- Require a session nonce for privileged localhost requests.
- Restrict CORS to known application origins.
- Validate content type.
- Enforce request-size limits.
- Never expose arbitrary filesystem browsing remotely.
- Never run shell commands received from cloud.
- Pipeline key must map to an installed allowlisted pipeline.
- File paths remain local unless user explicitly syncs them.

19.3 Job Security

Cloud job contains declarative configuration.

Allowed:

```
{
  "pipeline": "standard-single-camera",
  "capture_id": "uuid",
  "steps": ["pose", "metrics"]
}
```

Forbidden:

```
{
  "command": "arbitrary shell command"
}
```


19.4 Artifact Security

- Signed URLs for private cloud artifacts.
- Short URL expiry.
- No publicly accessible personal buckets.
- Validate MIME type and checksum.
- Sanitize export filenames.
- Strip unnecessary metadata from public reports.

19.5 Secret Management

Browser

May contain only public Supabase configuration.

Next.js Server

May contain:

- Supabase service role.
- Optional signing secrets.

Local Agent

May contain:

- Device credential.
- User-approved storage credential.
- Optional user-provided AI API key.

Repository

Must contain no active credential.

19.6 Model Supply-Chain Security

- Pin model source.
 - Record checksum.
 - Record license.
 - Avoid loading arbitrary Python pickle from untrusted users.
 - Prefer safe weight formats where supported.
 - Run research models only in explicitly enabled research mode.
-

20. Privacy Architecture

20.1 Default Privacy

- Session private.
- Video local.
- Cloud sync minimal.
- No training-data use.
- No public share link.
- No automatic facial analysis.

20.2 Training Consent

Consent is stored separately for:

- Product use.
- Cloud storage.
- Model training.
- Anonymous benchmark.
- Public demonstration.
- Coach sharing.

A user agreeing to cloud sync does not automatically agree to model training.

20.3 Data Deletion

User deletion flow must separately address:

- Cloud metadata.
- Cloud artifacts.
- Local metadata.
- Local artifacts.
- Original local video.
- Training dataset copies.

Training dataset deletion may require dataset-version invalidation and model-governance documentation.

20.4 Multi-Person Video

Before processing match footage:

- Display privacy notice.
- Offer face blur for exported reports later.
- Avoid uploading raw video by default.
- Store only required analytical outputs.

21. Observability Without Paid Services

21.1 Local Structured Logs

Use JSON Lines:

```
{
  "timestamp": "ISO-8601",
  "level": "INFO",
  "analysis_run_id": "uuid",
  "job_id": "uuid",
  "step": "pose_estimation",
  "event": "step_progress",
  "progress": 0.45
}
```

21.2 Log Rotation

- Maximum file size.
- Maximum retained days.
- Maximum retained files.
- Optional debug mode.
- User-controlled deletion.

21.3 Cloud Events

Store only important events:

- Job claimed.
- Step failed.
- Run completed.
- Worker incompatible.
- Sync failed.

Do not stream every processed frame into cloud logs.

21.4 Diagnostics Package

User can export:

```
diagnostics.zip
├─ system-info.json
```

```
|— worker-version.json
|— recent-logs.jsonl
|— job-summary.json
└— redacted-config.json
```

Diagnostics must remove:

- Access tokens.
- Signed URLs.
- Personal local paths where unnecessary.
- User video.
- API keys.

22. Performance Architecture

22.1 Processing Strategy

- Decode video once where practical.
- Batch pose inference.
- Crop player region.
- Cache normalized media.
- Reuse player track.
- Separate lightweight preview from detailed analysis.
- Avoid rendering overlays until metrics are valid.
- Allow step-level frame range.

22.2 Resource Profiles

Low Resource

- CPU only.
- Reduced resolution.
- Reduced frame sampling.
- Short clips.

Standard

- CPU or mid-range GPU.
- Full frame rate for target range.
- Medium pose model.

High Performance

- Dedicated GPU.
- Racket and shuttle tracking.

- Multi-camera.
- Detailed rendering.

22.3 Disk Guard

Before running:

```
required temporary space  
+ estimated output size  
+ safety margin
```

must be less than available disk.

22.4 Memory Guard

Worker estimates frame-batch size from:

- Resolution.
- Model.
- Device memory.
- Concurrent stages.

When out-of-memory occurs:

- Reduce batch size.
- Retry step.
- Record fallback.
- Do not silently change model precision without recording it.

23. Version Compatibility

23.1 Versioned Components

- Web application.
- Local Agent.
- Pipeline.
- Analysis schema.
- Artifact schema.
- Pose adapter.
- Model weights.
- Metric catalogue.
- Assessment rubric.
- Database schema.

23.2 Compatibility Response

GET /version returns:

```
{
  "agent_version": "0.1.0",
  "artifact_schema_versions": ["1.0.0"],
  "pipeline_contract_versions": ["1.0.0"],
  "capabilities": [
    "media.inspect",
    "pose.mediapipe",
    "metrics.basic"
  ]
}
```

23.3 Compatibility States

- Compatible.
- Compatible with reduced capability.
- Agent update required.
- Web update required.
- Artifact migration required.
- Unsupported legacy artifact.

23.4 Artifact Migration

Migration creates a new package revision.

Original package remains available until user deletes it.

24. Cost Guardrails

24.1 Mandatory-Free Components

Component	Initial choice	Required cost
Web hosting	Vercel Hobby	Rp0
Authentication	Supabase Free	Rp0
PostgreSQL	Supabase Free	Rp0
Video compute	Local machine	Rp0 cloud

Component	Initial choice	Required cost
Video storage	Local disk	Rp0 cloud
Metrics storage	Supabase Free	Rp0
LLM	Disabled/templates	Rp0
Monitoring	Local logs	Rp0

24.2 Quota Protection

Application settings include:

- Maximum evidence image size.
- Maximum thumbnails per session.
- Maximum cloud artifact size.
- Automatic local frame cleanup.
- Cloud-sync disabled by default.
- No automatic original-video upload.
- No automatic historical re-analysis.
- No background LLM call.
- No persistent GPU.

24.3 Database Budget

Target use for initial personal project:

Sessions and metadata:	< 10 MB
Metrics and findings:	< 50 MB
Annotations:	< 50 MB
Job and processing history:	< 50 MB
Registry and configuration:	< 10 MB
Safety reserve:	> 300 MB

These are internal targets, not provider guarantees.

24.4 Storage Budget

Supabase Storage should contain only:

- Avatar.
- Small thumbnail.
- Selected evidence frame.
- Small report.
- Optional compact manifest.

Raw video and dense artifacts stay local.

24.5 R2 Activation Rule

R2 may only be enabled after:

- User adds account deliberately.
 - Usage dashboard exists.
 - File-size limits exist.
 - User understands that overage may be billable.
 - Original-video synchronization remains opt-in.
-

25. Failure and Recovery Scenarios

25.1 Internet Disconnects During Analysis

- Local processing continues.
- Progress remains in SQLite.
- Cloud events enter sync outbox.
- Results synchronize later.

25.2 Supabase Project Is Paused

- Local Agent remains usable.
- Local sessions remain accessible.
- Cloud login or sync may be unavailable.
- Export package remains available.
- On reconnection, outbox resumes.

25.3 Browser Is Closed

- Local job continues if Local Agent remains running.
- User can reopen dashboard.
- Worker state is recovered from local database.

25.4 Local Agent Crashes

- Running step marked interrupted locally.
- Lease eventually expires.
- On restart, worker inspects incomplete outputs.
- Valid completed artifacts are retained.
- Step resumes or restarts according to its contract.

25.5 Source Video Is Moved

- Existing analysis remains accessible.
- New rendering requiring source media is blocked.
- User is prompted to relink the video.
- Fingerprint validates replacement.

25.6 Model Is Removed

- Existing results remain viewable.
- Re-analysis is unavailable.
- Model registry identifies missing dependency.
- User can reinstall matching checksum.

25.7 Cloud Sync Fails

- Analysis run remains valid locally.
- Sync error is shown separately from analysis status.
- Retry does not rerun analysis.

25.8 Insufficient Disk

- Job does not start or pauses safely.
- User sees estimated required space.
- Cache cleanup is offered.
- Original video is never deleted automatically.

26. Initial API Boundaries

26.1 Web-to-Cloud

Examples:

```
POST /api/devices/pair
POST /api/jobs
POST /api/analysis-runs
POST /api/artifacts/register
POST /api/sync/push
GET /api/sync/pull
```

Where possible, regular user operations may use Supabase directly under RLS. Privileged flows use server endpoints.

26.2 Agent-to-Cloud

```
POST /worker/auth/challenge
POST /worker/heartbeat
POST /worker/jobs/claim
POST /worker/jobs/{id}/progress
POST /worker/jobs/{id}/complete
POST /worker/jobs/{id}/fail
POST /worker/artifacts/register
```

These may initially be implemented as:

- PostgreSQL RPC.
- Next.js API routes.
- Supabase Edge Functions.

The contract must remain independent from the implementation.

26.3 Web-to-Local Agent

Initial use should be minimal.

Possible functions:

- Health check.
- Open local agent.
- Start linked session.
- Read progress.
- Open artifact directory.

A standalone local control page remains the fallback when direct browser-agent communication is blocked or incompatible.

27. Recommended Repository Architecture

```
badminton-motion-lab/
├─ apps/
│  ├─ web/
│  └─ research-admin/
├─ services/
│  └─ control-api/
└─
```

```
├─ workers/
│   └─ local-analyzer/
│       ├── src/
│       │   ├── agent/
│       │   ├── orchestration/
│       │   ├── steps/
│       │   ├── adapters/
│       │   ├── storage/
│       │   ├── sync/
│       │   └─ security/
│       ├── tests/
│       └─ pyproject.toml
├─ packages/
│   ├── domain/
│   ├── analysis-contracts/
│   ├── artifact-schema/
│   ├── badminton-ontology/
│   ├── coordinate-systems/
│   ├── metric-definitions/
│   ├── database/
│   └─ ui/
├─ ml/
│   ├── adapters/
│   ├── model-manifests/
│   ├── training/
│   ├── evaluation/
│   └─ experiments/
├─ validation/
│   ├── benchmark-manifests/
│   ├── fixtures/
│   └─ reports/
├─ infra/
│   ├── supabase/
│   │   ├── migrations/
│   │   ├── policies/
│   │   └─ seed/
│   └─ vercel/
├─ docs/
│   ├── product/
│   ├── architecture/
│   └─ adr/
```

```
| | | ontology/
| | | metrics/
| | | security/
| | | validation/
| |
| | scripts/
| | .github/
| | README.md
```

28. Testing Architecture

28.1 Unit Tests

Required for:

- Metric functions.
- Coordinate transforms.
- Manifest validation.
- Hashing.
- Event calculations.
- Assessment rules.
- Job transitions.

28.2 Contract Tests

Verify compatibility between:

- Web and cloud.
- Worker and cloud.
- Pipeline step and artifact schema.
- Model adapter and canonical output.
- Analysis package importer and exporter.

28.3 Integration Tests

Scenarios:

- Create session and local capture.
- Claim job.
- Run mock pipeline.
- Upload metric summary.
- Disconnect and reconnect.
- Cancel job.

- Retry failed step.
- Import analysis package.
- Relink moved video.

28.4 Golden Tests

A small fixed fixture set verifies:

- Exact manifest output.
- Metric results.
- Event calculations.
- Overlay coordinate placement.
- Database migration compatibility.

28.5 Performance Tests

Track:

- Decode speed.
- Pose FPS.
- Peak RAM.
- Peak GPU memory.
- Temporary disk.
- Artifact size.
- Database row growth.
- Sync payload size.

28.6 Security Tests

- RLS isolation.
 - Revoked worker.
 - Expired lease.
 - Invalid device signature.
 - Path traversal bundle.
 - Oversized archive.
 - Invalid model checksum.
 - Unauthorized artifact access.
-

29. Implementation Sequence

Architecture Sprint A — Repository and Contracts

Deliverables

- Monorepo.
- Shared IDs and enums.
- Artifact schema.
- Pipeline-step contract.
- Metric-observation contract.
- Basic CI.
- ADR files.
- Supabase local migrations.

Exit Gate

TypeScript and Python can validate the same example analysis manifest.

Architecture Sprint B — Local Agent Foundation

Deliverables

- Local SQLite.
- Data directory.
- `GET /health`.
- Media selection.
- FFmpeg inspection.
- Media fingerprint.
- Local capture record.
- Structured logs.
- Analysis-package skeleton.

Exit Gate

A video can be registered locally and exported as a valid empty analysis package.

Architecture Sprint C — Cloud Control Plane

Deliverables

- Supabase Auth.
- Users and athletes.
- Sessions and captures.

- Worker devices.
- Analysis jobs.
- Analysis runs.
- RLS.
- Web session dashboard.

Exit Gate

A signed-in user can create a session and link one registered Local Agent.

Architecture Sprint D — Job Orchestration

Deliverables

- Job creation.
- Atomic job claim.
- Lease.
- Heartbeat.
- Progress events.
- Cancellation.
- Retry.
- Offline outbox.

Exit Gate

A mock analysis can continue when browser closes and synchronize after reconnecting.

Architecture Sprint E — Baseline Analysis

Deliverables

- Video normalization.
- Proxy generation.
- Baseline pose adapter.
- Pose artifact.
- Skeleton viewer.
- Manual stroke range.
- Manual contact frame.
- Basic angles.
- CSV and JSON export.

Exit Gate

One local video produces visible, versioned, evidence-linked metrics at Rp0 cloud compute cost.

Architecture Sprint F — Hardening

Deliverables

- Diagnostics.
- Bundle integrity.
- Security tests.
- Artifact cleanup.
- Version compatibility.
- Backup/export.
- Failure recovery tests.

Exit Gate

The platform survives expected disconnection, restart, moved-file, and failed-step scenarios without losing the original analysis.

30. Architecture Definition of Done

Architecture v0.1 is considered successfully implemented when:

1. Web application can create athlete, session, and capture metadata.
 2. Local Agent can register and link to the authenticated user.
 3. Video selection occurs locally.
 4. Original video is not automatically uploaded.
 5. Local Agent can claim or receive an analysis job.
 6. Job can continue after browser is closed.
 7. Pipeline steps have versioned input and output manifests.
 8. Analysis produces a portable package.
 9. Dense pose data remains outside PostgreSQL.
 10. Summary metrics synchronize to Supabase.
 11. User can review skeleton and evidence frames.
 12. User can correct contact frame without overwriting model output.
 13. Re-analysis creates a new analysis run.
 14. Partial reprocessing is structurally supported.
 15. Failed cloud synchronization does not invalidate local analysis.
 16. Local Agent can operate during internet interruption.
 17. Database RLS prevents cross-user access.
 18. Service-role credentials never exist in browser or Local Agent.
 19. Imported bundles are checksum and path validated.
 20. Mandatory recurring cloud cost remains Rp0.
 21. Architecture supports future local, self-hosted, and cloud workers through the same contract.
 22. Multi-camera, racket, shuttle, rally, and coach entities can be added without redesigning the execution model.
-

31. Final Architecture Position

The recommended system is not a conventional web application that sends a video to an API.

It is a hybrid platform with:

- A cloud control plane for identity, metadata, coordination, and history.
- A local execution plane for private and cost-free video processing.
- A portable artifact format for reproducibility and research.
- A deterministic analytical core.
- Optional cloud services that remain replaceable and disabled by default.

This structure costs slightly more engineering effort at the beginning than directly uploading video to a hosted inference endpoint. However, it prevents the project from becoming:

- Expensive to operate.
- Dependent on one cloud provider.
- Unable to work offline.
- Difficult to validate.
- Unsafe for private videos.
- Impossible to reproduce.
- Difficult to expand into a serious research platform.